

دانشگاه صنعتی اصفهان

دانشکده مهندسی برق و کامپیوتر

پروپزال پروژه کارشناسی ارشد

نام و نام خانوادگی: نوید نصیری

شماره دانشجویی: ۴۰۲۱۴۳۹۴

استاد راهنما: دکتر مسعود هاشمی

۱ سوال اول

۱.۱ بخش اول

اکسترنال فرگمنتیشن به حافظه های خارج از سگمنت ها گفته میشود که قابل استفاده و دیدن است برای گرفتن حافظه اما اینترنال فرگمنتیشن به حافظه های اضافه ای گفته میشود که در داخل هر سگمنت ممکن است ایجاد شود و دیگر قابل استفاده برای یک سگمنت دیگر نیست.

۲.۱ بخش دوم

حافظه ها انواع مختلفی دارند. رجیستر ها که درون سی پی یو هستند و سرعت بسیاری دارند اما مقدار حافظه کمی را شامل میشود و گران قیمت هستند و حد اکثر تا ۱کیلو بایت هستند. پس از آن حافظه کش است که باز هم سرعت بالایی دارد (اما کمتر از رجیستر)و حداکثر تا ۱۶ مگابایت است و قیمت بالایی دارد. پس از آن حافظه ممری اصلی است که تا ۶۴ گیگابایتت است و سرعت کمتری نسبت به قبلی ها دارد. و در آخر هم حافظه دیسک است که سرعت کمی دارد اما تا ۱۰۰ گیگا بایت ظرفیت میتواند داشته باشد. حال برای اینکه حافظه های با سرعت خیلی بالا اگر به حجم زیادی نیاز داشته باشیم بسیار گران خواهد شد و از طرفی ما هر چه به سی پی یو نزدیک تر میشویم نیاز داریم که سرعت حافظه بالاتری داشته باشیم، پس با لایه بندی کردن حافظه میتوانیم با سرعت مناسبی با سی پی یو تبادل داشته باشیم و از طرفی حافظه کافی هم برای ذخیره اطلاعاتمان در دسترسمان باشد.

۳.۱ بخش سوم

برای اینکه از حافظه اختصاص داده شده بتوان بهترین استفاده را کرد و چون هیچ پیشبینی ای نسبت به این نداریم که هر پروسه قرار است چه مقدار حافظه استک یا هیپ اشغال کند هر دو حافظه را از یک سمت شروع نمیکنیم. برای حل این مشکل استک را از بالا و هیپ را از پایین شروع به پر شدن میکنند. اگر پشت سر هم قرار میگرفتند ممکن بود با پر شدن هیپ داده ها روی استک ها اوررایت شود و داده های استک را از دست بدهیم.

۴.۱ بخش چهارم

به این صورت که توسط سیستم عامل کد مربوط به آن بخش از کد که بین چند پروسس مشترک است به همان بخش در حافظه فیزیکی مپ میشود. یعنی درواقع تمام پروسه هایی که از کتابخانه خاصی استفاده میکنند، بخش مربوط به کد در حافظه مجازی آنها به یک مکان در حافظه فیزیکی مپ میشود که آن کد در آن وجود دارد. لازم به پشبیبانی های سخت افزاری داریم . باید بخش کد در کد شیرینگ تنها دسترسی خواندن و اجرا شدن داشته باشد و دسترسی نوشتن در آن نباشد.اما قسمت های هیپ و استک باید قابلیت نوشتن و خواندن داشته باشند. پس اگر یک برنامه بخواهد چیزی در قسمت کد بنویسد، سخت افزار با دادن یک اکسپشن اجازه آن را نمیدهد.س

۵.۱ بخش پنجم

بزرگ گرفتن اندازه پیجینگ ها باعث ایجاد مشکل internal fragmentation میشود و باعث میشود نتوان به صورت بهینه از فضای گرفته شده استفاده کرد.و کوچک گرفتن اندازه پیجینگ ها باعث میشود که ارایه پیج تیبل بزرگی ایجاد شود که خود نیاز به داشتن حافظه به اندازه کافی دارد.

۶.۱ بخش ششم

برای اینکه لازم است سیستم عامل با دریافت این اکسپشن از سمت سخت افزار به هارد دیسک برود و صفحه مربوطه را بیابد و آن را به حافظه اصلی انتقال دهد و بیت present را هم یک قرار دهد

۷.۱ بخش هفتم

سی پی یو و سخت افزار برای یافتن آرایه پیج تیبل نیاز دارند تا یک ادرس به اول آن ارایه را داشته باشند که این ادرس در رجیستر CR3 قرار میگیرد.

۸.۱ بخش هشتم

TLB Miss & No Page Fault

زمانی TLB miss رخ میدهد که آدرس خواسته شده در جدول TLB نباشد در نتیجه به حافظه اصلی نیاز داریم. حال اگر صفحه مورد نیاز در حافظه اصلی یافت شدو دی این صورت Page Fault رخ نمیدهد.

TLB MIss & Page Fault

زمانی که آدرس خواسته شده در جدول TLB نباشد و پس از مراجعه به حافظه اصلی باز هم صفحه مورد نیاز یافت نشود در این صورت Page Fault رخ میدهد و به هارد دیسک مراجعه میشود.

TLB Hit & No Page Fault

زمانی که TLB hit رخ میدهد آدرس مورد نیاز درون جدول TLB یافت شده است و دیگر به حافظه اصلی نیازی نیست و Page Fault هم رخ نخواهد داد.

TLB Hit & Page Fault

این حافت رخ نمیدهد چون در صورتی که TLB Hit رخ بدهد نیازی نداریم دیگر به حافظه اصلی دسترسی داشته باشیم که Page Fault رخ دهد. پس در این حافت Page Fault نمیتواند رخ بدهد.

۲ سوال دوم

State(after)	Hit	Access
۲	NO	۲
۲،۳	NO	۳
۲،۳،۱	NO	۱
۲،۳،۱،۵	NO	۵
۲،۱،۵،۶	NO	۶
۲،۱،۵،۶	YES	۲
۲،۱،۵،۶	YES	۱
۲،۱،۵،۶	YES	۵
۲،۵،۶،۳	NO	۳
۲،۵،۶،۳	YES	۲
۲،۵،۶،۳	YES	۶
۲،۵،۶،۳	YES	۵
۲،۶،۳،۴	NO	۴
۲،۶،۳،۴	YES	۳
۲،۶،۳،۴	YES	۲

جدول ۱: Optimal

State(after)	Hit	Access
۲	NO	۲
۲،۳	NO	۳
۲،۳،۱	NO	۱
۲،۳،۱،۵	NO	۵
۳،۱،۵،۶	NO	۶
۱،۵،۶،۲	NO	۲
۱،۵،۶،۲	YES	۱
۱،۵،۶،۲	YES	۵
۱،۵،۲،۳	NO	۳
۱،۵،۲،۳	YES	۲
۱،۵،۲،۶	NO	۶
۱،۵،۲،۶	YES	۵
۱،۵،۲،۴	NO	۴
۱،۵،۲،۳	NO	۳
۱،۵،۲،۳	YES	۲

جدول ۲: LFU

State(after)	Hit	Access
۲	NO	۲
۲،۳	NO	۳
۲،۳،۱	NO	۱
۲،۳،۱،۵	NO	۵
۳،۱،۵،۶	NO	۶
۱،۵،۶،۲	NO	۲
۱،۵،۶،۲	YES	۱
۱،۵،۶،۲	YES	۵
۱،۵،۲،۳	NO	۳
۱،۵،۲،۳	YES	۲
۵،۲،۳،۶	NO	۶
۵،۲،۳،۶	YES	۵
۵،۲،۶،۴	NO	۴
۵،۶،۴،۳	NO	۳
۵،۴،۳،۲	NO	۲

جدول ۳: LRU

۳ سوال سوم

در این سوال برای یافتن میانگین زمان دسترسی به حافظه در دو حالت درنظر میگیریم.
حالت اول اینکه TLB miss رخ دهد و سپس دچار page fault شویم. دی این حالت محاسبات زیر را خواهیم داشت.

$$T_{\text{TLB}} + (P_{\text{TLBmiss}} \times P_{\text{PageFault}} \times T_{\text{Disk}} + P_{\text{TLBmiss}} \times 2 \times T_{\text{DRAM}}) = 1 + (0.01 \times 2 \times 10^{-5} \times 10^{7} + 0.01 \times 2 \times 100) = 5ns$$

در حالت دوم حالتی را در نظر میگیریم TLBmiss رخ نمیدهدو وارد cache میشویم . حال در اینجا ممکن است دو اتفاق بیوفتد. یا اینکه cache miss رخ دهد و یا اینکه داده ای که نیاز داریم در cache موجود باشد. برای محاسبات این بخش داریم :

$$1 + (0.99 \times 1 + \times 0.99 \times 0.01 \times 100) = 2.98ns$$

و در آخر دو مقدار بالا را جمع میکنیم :

$$2.98 + 5 = 7.98ns$$

۴ سوال چهارم

۱.۴ قسمت اول

عدد مورد نظر برابر است با 0x45d برای دسترسی به این آدرس ابتدا به صورت باینری مینویسیم. 010001011101

$$2^5 = 32 => VPN = 5$$

در این عدد دو بیت سمت چپ نشان دهنده SN است . پنج بیت بعدی برابر VPN است . و پنج بیت باقی مانده هم برابر با offset است.

با توجه به جدول میتوانیم آدرس PTE رابدست آوریم:

$$addressPTE = Base[SN] + (VPN \times sizeof(PTE)) = 64 + (2 \times 1) = 66$$

حال آدرس PTE را پیدا کردیم.برای یافتن مقدار PTE باید مقدار ۶۶ را بر ۳۲ تقسیم کنیم. سپس مقدار خارج قسمت برابر PFN و مقدار باقی مانده برابر index است. در نتیجه مقدار PFN برابر خواهد بود با 2 و مقدار اندیس آن برابر میشود با 2. حال با توجه به جدولی که داده شده است میتوانیم در PFN = 2 اندیس دوم را بیابیم که برابر است با c3 .پس در واقع درون این PTE مقدار باینری 11000011 قرار میگیرد که بیت سمت چپ آن یا بیت ولید آن برابر ۱ است که نشان دهنده این است که این آدرس به این پروسه تخصیص داده شده است. و سایر بیت ها نشان دهنده مقدار PPN است که برابر 67 است. پس ما شماره صفحه را یافتیم. حال به افست یعنی مقدار ۲۹ است مراجعه میکنیم . که مقدار ان ۴۰ هگز است.

پس برای این دو دسترسی به حافظه انجام دادیم. که یکی صفحه ۲ اندیس ۲ و یکی صفحه صفحه ۶۷ و اندیس ۲۹ است.

۲.۴ قسمت دوم

عدد مورد نظر برابر است با 0xc85 برای دسترسی به این آدرس ابتدا به صورت باینری مینویسیم. 110010000101

در این عدد دو بیت سمت چپ نشان دهنده SN است . پنج بیت بعدی برابر VPN است . و پنج بیت باقی مانده هم برابر با offset است.

با توجه به جدول میتوانیم آدرس PTE رابدست آوریم:

$$addressPTE = Base[SN] + (VPN \times sizeof(PTE)) = 640 + (4 \times 1) = 644$$

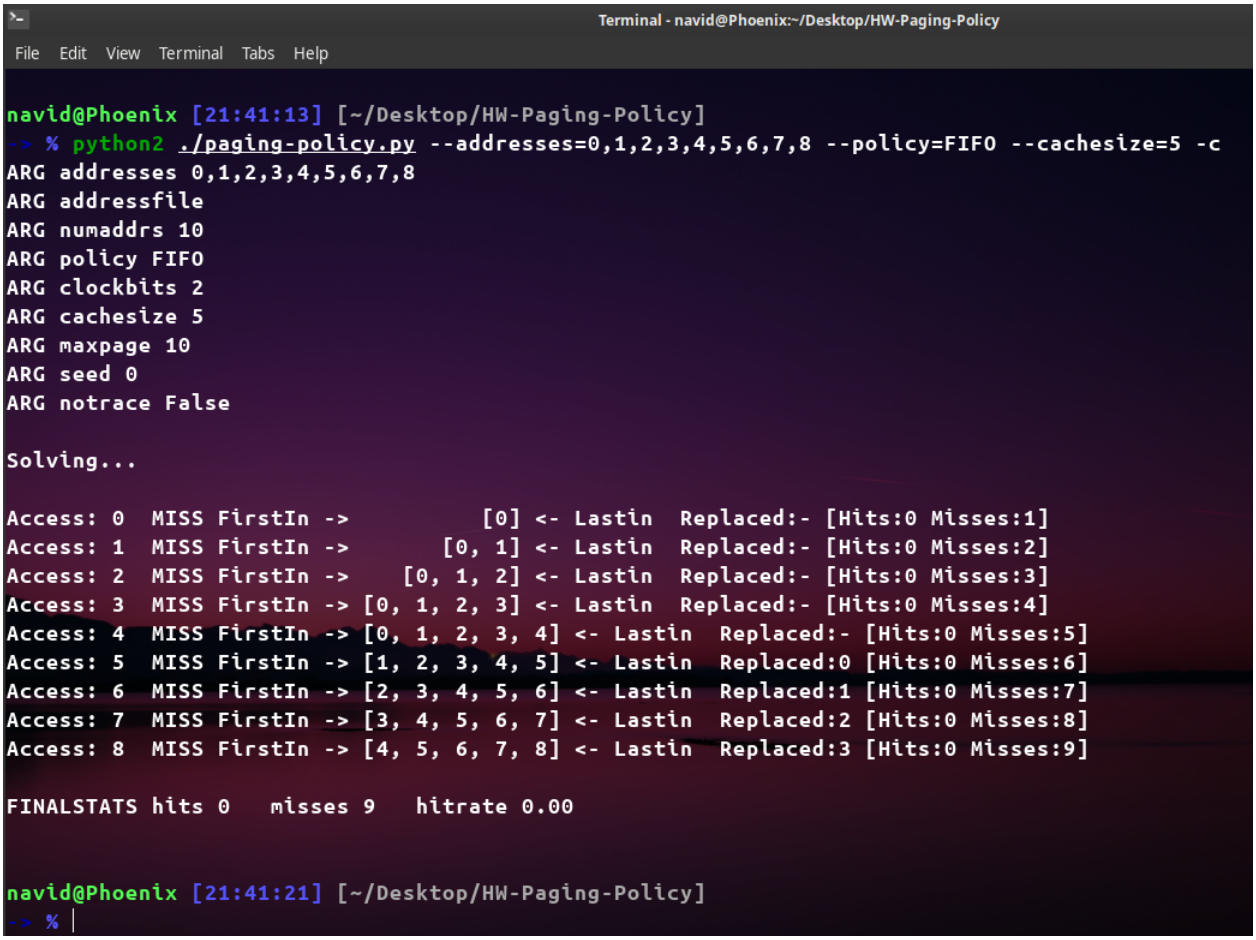
حال باز هم مثل قسمت اول عمل میکنیم و داریم PFN برابر با ۲۰ و مقدار اندیس آن برابر میشود با ۴.

حال با توجه به جدول داده شده در سوال میتوانیم به مقدار 20 که این مقدار را اگر به صورت باینری بنویسیم خواهیم داشت 00100000که در این عدد بیت سمت چپ یا بیت ولید برابر با صفر است که نتیجه میدهد این مقدار نشان دهنده هیچ آدرس فیزیکالی نیست.

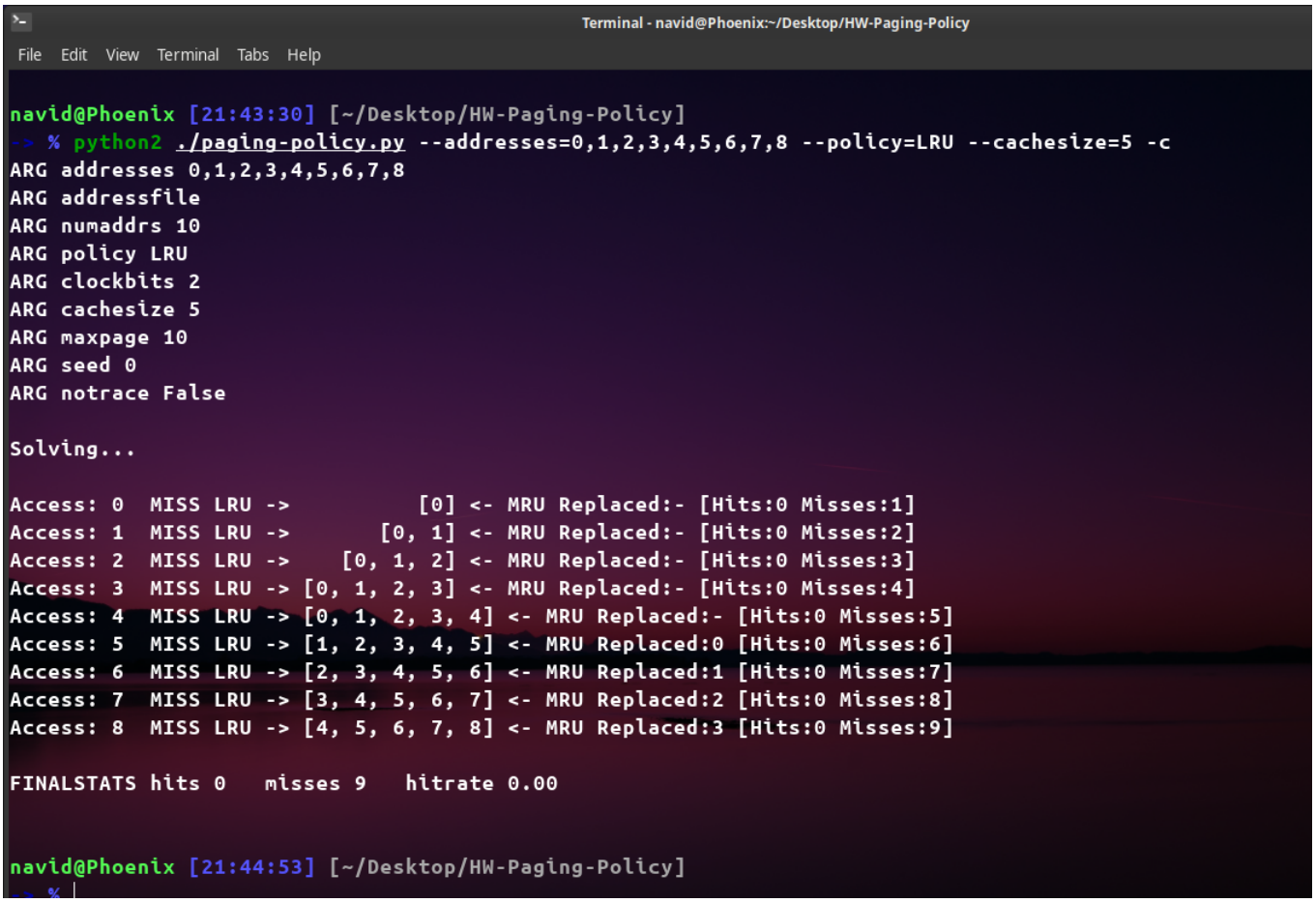
۵ سوال پنجم

۶ سوال ششم

در روش FIFO اینگونه عمل میکنیم که آدرسی که اول وارد شده باشد آخر از همه خارج میشود یعنی آدرسی را از کش خارج میکنیم که دیر تر از همه وارد کش شده باشد.در اینجا با دادن آدرس های نمایش داده شده در عکس را بدهیم در همه مراحی به MISS برخورد میکنیم.حداقل مقدار حافظه کش که میتواند کمک کند به تعداد ادرس های داده شده یعنی ۹ تا باید باشد.



در این روش آن آدرسی که اخیرا کمتر استفاده شده است را بیرون میندازیم.و در اینجا اگر در آدرس های داده شده ادرس تکراری نباشد میتوانیم بد ترین حالت را داشته باشیم. برای اینکه بتوانیم برای این حالت بیشترین تعداد hit را داشته باشیم باید به تعداد آدرس داده شده کش داشته باشیم یعنی ۹.



در روش MRU آن آدرسی را بیرون می اندازیم که اخیرا بیشترین تکرار را داشته باشد. در مثال عکس هم بعد از امدن همه ادرس ها میتوانیم بین دو آدرس آخر تغییر دهیم تا همین روند Miss ادامه پیدا کند. برای اینکه بتوانیم بیشترین تعداد hit را داشته باشیم باید باز هم به تعداد آدرس داده ها باشد یعنی در اینجا چون آدرس هایمان بین ۰ تا ۸ است پس اگر مقدار حافظه کش ۹ باشد بیشتری مقدار hit رخ خواهد داد.

